

NUMERICAL ANALYSIS & COMPUTING IN PYTHON

A Complete Practical Handbook: Theoretical Formulations, Step-by-Step
Procedural Implementations & Iteration Convergence Tables

20 Core Numerical Methods Included

- Roots of Non-Linear Equations
- Systems of Linear Equations
- Interpolation & Polynomials
- Numerical Quadrature
- Ordinary Differential Equations
- Eigenvalue Problems & Fitting

Featuring Clean Python 3 Syntax, Pure Procedural Functions, and Formatted Iteration Tables

Code by Anushka

Semester 5 Mathematics & Numerical Computing Laboratory Reference

September 22, 2026

Contents

1	Transcendental & Polynomial Equations	2
1.1	Bisection Method	2
1.2	Regula-Falsi (False Position) Method	4
1.3	Newton-Raphson Method	5
1.4	Secant Method	6
1.5	Fixed-Point Iteration Method	7
2	Systems of Linear Algebraic Equations	8
2.1	Gauss Elimination Method	8
2.2	Gauss-Jordan Elimination Method	10
2.3	Gauss-Jacobi Iterative Method	11
2.4	Gauss-Seidel Iterative Method	13
3	Interpolation & Polynomial Approximation	15
3.1	Lagrange Interpolation	15
3.2	Newton's Divided Difference Interpolation	16
3.3	Newton's Forward Difference Interpolation	17
4	Numerical Quadrature & Integration	18
4.1	Trapezoidal Rule of Numerical Integration	18
4.2	Simpson's 1/3 Rule of Numerical Integration	20
4.3	Simpson's 3/8 Rule of Numerical Integration	21
5	Ordinary Differential Equations (ODEs)	22
5.1	Euler's Method for ODEs	22
5.2	Modified Euler's (Heun's) Method	23
5.3	Runge-Kutta 4th Order Method (RK4)	24
6	Eigenvalue Problems & Curve Fitting	25
6.1	Power Method for Dominant Eigenvalue	25
6.2	Linear Curve Fitting (Least Squares)	27
7	Master Summary & Quick-Reference Guide	28

Chapter 1

Transcendental & Polynomial Equations

1.1 Bisection Method

Category: [Roots of Non-Linear Equations](#) — **Code by:** [Anushka](#) — **Source:** `01_bisection_detailed.py`

The Bisection method is a robust bracketing method for finding real roots of a continuous function $f(x) = 0$. Given an initial bracket $[a, b]$ where $f(a) \cdot f(b) < 0$, Bolzano's theorem guarantees at least one root in (a, b) . In each iteration, the midpoint c is calculated and the sign of $f(c)$ determines which sub-interval is retained.

Mathematical Theory & Formulation

$$c = \frac{a+b}{2.0}, \quad \text{Update: } b = c \text{ if } f(a) \cdot f(c) < 0, \quad a = c \text{ if } f(a) \cdot f(c) > 0$$

$$\text{Convergence: Linear, } |e_{n+1}| \leq \frac{1}{2}|e_n|, \quad \text{Error Bound: } \frac{b-a}{2^n}$$

Python Implementation (Step-by-Step Loops & Pure Functions)

```
1 # Code by Anushka
2
3 def bisection(f, a, b, tol=1e-5, max_iter=50):
4     if f(a) * f(b) >= 0:
5         print("Error: f(a) and f(b) must have opposite signs.")
6         return None
7
8     print(f"'Iter':<6>{'a':<12>{'b':<12>{'c (Mid)':<14>{'f(c)':<12>}")
9     print("-" * 56)
10
11     for i in range(1, max_iter + 1):
12         c = (a + b) / 2.0
13         fc = f(c)
14         print(f"{i:<6}{a:<12.5f}{b:<12.5f}{c:<14.5f}{fc:<12.5f}")
15
16         if abs(fc) < tol or (b - a) / 2.0 < tol:
17             return c
18
19         if f(a) * fc < 0:
20             b = c
21         else:
22             a = c
23
24     return (a + b) / 2.0
25
26
27 def f(x):
28     return x**3 - x - 2
29
30
31 root = bisection(f, 1.0, 2.0)
32 print(f"\nRoot: {root:.5f}")
```

Program Execution Output & Convergence Table

Iter	a	b	c (Mid)	f(c)
1	1.00000	2.00000	1.50000	-0.12500
2	1.50000	2.00000	1.75000	1.60938
3	1.50000	1.75000	1.62500	0.66602
4	1.50000	1.62500	1.56250	0.25220
5	1.50000	1.56250	1.53125	0.05911
6	1.50000	1.53125	1.51562	-0.03405
7	1.51562	1.53125	1.52344	0.01225
8	1.51562	1.52344	1.51953	-0.01097
9	1.51953	1.52344	1.52148	0.00062
10	1.51953	1.52148	1.52051	-0.00518
11	1.52051	1.52148	1.52100	-0.00228
12	1.52100	1.52148	1.52124	-0.00083
13	1.52124	1.52148	1.52136	-0.00010
14	1.52136	1.52148	1.52142	0.00026
15	1.52136	1.52142	1.52139	0.00008
16	1.52136	1.52139	1.52138	-0.00001
17	1.52138	1.52139	1.52139	0.00003

Root: 1.52139

1.2 Regula-Falsi (False Position) Method

Category: Roots of Non-Linear Equations — **Code by:** Anushka — **Source:** 02_regula_falsi_detailed.py

The Regula-Falsi (False Position) method combines the reliability of bracketing with the acceleration of linear interpolation. Instead of bisecting the interval geometrically, it connects $(a, f(a))$ and $(b, f(b))$ with a secant line and determines the root estimate c at the line's x -intercept.

Mathematical Theory & Formulation

$$c = \frac{af(b) - bf(a)}{f(b) - f(a)} = a - \frac{f(a)(b - a)}{f(b) - f(a)}$$

Update: $b = c$ if $f(a) \cdot f(c) < 0$, $a = c$ if $f(a) \cdot f(c) > 0$

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 def regula_falsi(f, a, b, tol=1e-5, max_iter=50):
4     if f(a) * f(b) >= 0:
5         print("Error: f(a) and f(b) must have opposite signs.")
6         return None
7
8     print(f"{'Iter':<6}{ 'a':<12}{ 'b':<12}{ 'c (False)':<14}{ 'f(c)':<12}")
9     print("-" * 56)
10
11     c = a
12     for i in range(1, max_iter + 1):
13         fa = f(a)
14         fb = f(b)
15         c = (a * fb - b * fa) / (fb - fa)
16         fc = f(c)
17         print(f"{'i':<6}{ 'a':<12.5f}{ 'b':<12.5f}{ 'c':<14.5f}{ 'fc':<12.5f}")
18
19         if abs(fc) < tol:
20             return c
21
22         if fa * fc < 0:
23             b = c
24         else:
25             a = c
26
27     return c
28
29
30 def f(x):
31     return x**3 - 2 * x - 5
32
33
34 root = regula_falsi(f, 2.0, 3.0)
35 print(f"\nRoot: {root:.5f}")

```

Program Execution Output & Convergence Table

Iter	a	b	c (False)	f(c)
1	2.00000	3.00000	2.05882	-0.39080
2	2.05882	3.00000	2.08126	-0.14720
3	2.08126	3.00000	2.08964	-0.05468
4	2.08964	3.00000	2.09274	-0.02020
5	2.09274	3.00000	2.09388	-0.00745
6	2.09388	3.00000	2.09431	-0.00275
7	2.09431	3.00000	2.09446	-0.00101
8	2.09446	3.00000	2.09452	-0.00037
9	2.09452	3.00000	2.09454	-0.00014
10	2.09454	3.00000	2.09455	-0.00005
11	2.09455	3.00000	2.09455	-0.00002
12	2.09455	3.00000	2.09455	-0.00001

Root: 2.09455

1.3 Newton-Raphson Method

Category: Roots of Non-Linear Equations — **Code by:** Anushka — **Source:** 03_newton_raphson_detailed.py

The Newton-Raphson method is an open root-finding technique that utilizes the tangent line at the current estimate x_k . Starting from an initial guess x_0 , the method converges with quadratic order near simple roots.

Mathematical Theory & Formulation

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

Order of Convergence: $p = 2$ (Quadratic, requires $f'(x_k) \neq 0$)

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 def newton_raphson(f, df, x0, tol=1e-5, max_iter=50):
4     print(f"{'Iter':<6}{ 'x0':<12}{ 'f(x0)':<14}{ 'df(x0)':<14}{ 'x1':<12}")
5     print("-" * 58)
6
7     for i in range(1, max_iter + 1):
8         fx = f(x0)
9         dfx = df(x0)
10        if dfx == 0:
11            print("Error: Derivative is zero.")
12            return None
13
14        x1 = x0 - (fx / dfx)
15        print(f"{'i':<6}{x0:<12.5f}{fx:<14.5f}{dfx:<14.5f}{x1:<12.5f}")
16
17        if abs(x1 - x0) < tol or abs(f(x1)) < tol:
18            return x1
19
20        x0 = x1
21
22    return x0
23
24
25 def f(x):
26     return x**3 - 3 * x + 1
27
28
29 def df(x):
30     return 3 * x**2 - 3
31
32
33 root = newton_raphson(f, df, 1.5)
34 print(f"\nRoot: {root:.5f}")

```

Program Execution Output & Convergence Table

Iter	x0	f(x0)	df(x0)	x1
1	1.50000	-0.12500	3.75000	1.53333
2	1.53333	0.00504	4.05333	1.53209

Root: 1.53209

1.4 Secant Method

Category: Roots of Non-Linear Equations — **Code by:** Anushka — **Source:** 04_secant_detailed.py

The Secant method approximates the analytical derivative in Newton-Raphson using a finite-difference slope between the two most recent estimates x_{k-1} and x_k . This avoids calculating $f'(x)$ while retaining rapid super-linear convergence.

Mathematical Theory & Formulation

$$x_{k+1} = x_k - f(x_k) \cdot \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}$$

$$\text{Order of Convergence: } p = \frac{1 + \sqrt{5}}{2} \approx 1.618 \quad (\text{Golden Ratio, Superlinear})$$

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 def secant(f, x0, x1, tol=1e-5, max_iter=50):
4     print(f"{'Iter':<6}{'x0':<12}{'x1':<12}{'x2':<14}{'f(x2)':<12}")
5     print("-" * 56)
6
7     for i in range(1, max_iter + 1):
8         f0 = f(x0)
9         f1 = f(x1)
10        if f1 - f0 == 0:
11            print("Error: Division by zero in Secant formula.")
12            return None
13
14        x2 = x1 - f1 * (x1 - x0) / (f1 - f0)
15        f2 = f(x2)
16        print(f"{'i':<6}{'x0':<12.5f}{'x1':<12.5f}{'x2':<14.5f}{'f2':<12.5f}")
17
18        if abs(x2 - x1) < tol or abs(f2) < tol:
19            return x2
20
21        x0 = x1
22        x1 = x2
23
24    return x1
25
26
27 def f(x):
28     return x**2 - 4
29
30
31 root = secant(f, 1.0, 3.0)
32 print(f"\nRoot: {root:.5f}")

```

Program Execution Output & Convergence Table

Iter	x0	x1	x2	f(x2)
1	1.00000	3.00000	1.75000	-0.93750
2	3.00000	1.75000	1.94737	-0.20776
3	1.75000	1.94737	2.00356	0.01425
4	1.94737	2.00356	1.99995	-0.00019
5	2.00356	1.99995	2.00000	-0.00000

Root: 2.00000

1.5 Fixed-Point Iteration Method

Category: Roots of Non-Linear Equations — **Code by:** Anushka — **Source:** 05_fixed_point_detailed.py

Fixed-point iteration rewrites the root problem $f(x) = 0$ as $x = g(x)$. Starting from x_0 , successive estimates are generated by repeatedly evaluating $g(x)$. Convergence is guaranteed if $|g'(x)| < 1$ in an interval containing the fixed point.

Mathematical Theory & Formulation

$$x_{k+1} = g(x_k)$$

Convergence Condition: $|g'(x^*)| < 1$ (Contraction Mapping)

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 import math
4
5 def fixed_point(g, x0, tol=1e-5, max_iter=50):
6     print(f"{'Iter':<6}{'x0':<12}{'x1 = g(x0)':<16}{'Error':<12}")
7     print("-" * 46)
8
9     for i in range(1, max_iter + 1):
10         x1 = g(x0)
11         err = abs(x1 - x0)
12         print(f"{'i':<6}{'x0':<12.5f}{'x1':<16.5f}{'err':<12.5f}")
13
14         if err < tol:
15             return x1
16
17         x0 = x1
18
19     return x0
20
21
22 def g(x):
23     return math.cos(x)
24
25
26 root = fixed_point(g, 0.5)
27 print(f"\nFixed Point: {root:.5f}")

```

Program Execution Output & Convergence Table

Iter	x0	x1 = g(x0)	Error
1	0.50000	0.87758	0.37758
2	0.87758	0.63901	0.23857
3	0.63901	0.80269	0.16367
4	0.80269	0.69478	0.10791
5	0.69478	0.76820	0.07342
6	0.76820	0.71917	0.04903
7	0.71917	0.75236	0.03319
8	0.75236	0.73008	0.02227
9	0.73008	0.74512	0.01504
10	0.74512	0.73501	0.01011
11	0.73501	0.74183	0.00682
12	0.74183	0.73724	0.00459
13	0.73724	0.74033	0.00309
14	0.74033	0.73825	0.00208
15	0.73825	0.73965	0.00140
16	0.73965	0.73870	0.00095
17	0.73870	0.73934	0.00064
18	0.73934	0.73891	0.00043
19	0.73891	0.73920	0.00029
20	0.73920	0.73901	0.00019
21	0.73901	0.73914	0.00013
22	0.73914	0.73905	0.00009
23	0.73905	0.73911	0.00006
24	0.73911	0.73907	0.00004
25	0.73907	0.73910	0.00003
26	0.73910	0.73908	0.00002
27	0.73908	0.73909	0.00001
28	0.73909	0.73908	0.00001

Fixed Point: 0.73908

Chapter 2

Systems of Linear Algebraic Equations

2.1 Gauss Elimination Method

Category: [Systems of Linear Algebraic Equations](#) — **Code by:** [Anushka](#) — **Source:** [06_gauss_elimination_details](#)

Gauss Elimination is a direct method for solving a system of linear equations $Ax = b$. It transforms the augmented matrix $[A \mid b]$ into upper-triangular form through forward row operations, followed by backward substitution to determine the unknowns.

Mathematical Theory & Formulation

Forward Elimination: $m_{ik} = \frac{a_{ik}}{a_{kk}}, \quad R_i \leftarrow R_i - m_{ik}R_k \quad (i > k)$

Back Substitution: $x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j=i+1}^n a_{ij}x_j \right)$

Python Implementation (Step-by-Step Loops & Pure Functions)

```
1 # Code by Anushka
2
3 def gauss_elimination(A, b):
4     n = len(b)
5
6     # Forward Elimination
7     for i in range(n):
8         for j in range(i + 1, n):
9             ratio = A[j][i] / A[i][i]
10            for k in range(i, n):
11                A[j][k] = A[j][k] - ratio * A[i][k]
12            b[j] = b[j] - ratio * b[i]
13
14     # Back Substitution
15     x = []
16     for _ in range(n):
17         x.append(0.0)
18
19     for i in range(n - 1, -1, -1):
20         sum_ax = 0.0
21         for j in range(i + 1, n):
22             sum_ax = sum_ax + A[i][j] * x[j]
23         x[i] = (b[i] - sum_ax) / A[i][i]
24
25     return x
26
27
28 A = [
29     [2.0, 1.0, -1.0],
30     [-3.0, -1.0, 2.0],
31     [-2.0, 1.0, 2.0]
32 ]
33 b = [8.0, -11.0, -3.0]
34
35 ans = gauss_elimination(A, b)
36 for i in range(len(ans)):
37     print(f"x{i+1} = {ans[i]:.4f}")
```

Program Execution Output & Convergence Table

```
x1 = 2.0000  
x2 = 3.0000  
x3 = -1.0000
```

2.2 Gauss-Jordan Elimination Method

Category: Systems of Linear Algebraic Equations — **Code by:** Anushka — **Source:** 07_gauss_jordan_detailed.

Gauss-Jordan elimination reduces the coefficient matrix directly into reduced row echelon form (identity matrix I) by performing row eliminations both below and above each pivot element. The solution appears directly in the augmented column.

Mathematical Theory & Formulation

$$\text{Normalize Pivot Row: } R_i \leftarrow \frac{R_i}{a_{ii}}$$

$$\text{Eliminate Other Rows: } R_k \leftarrow R_k - a_{ki}R_i \quad (\forall k \neq i)$$

$$\text{Result: } [A \mid b] \longrightarrow [I \mid x]$$

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 def gauss_jordan(A, b):
4     n = len(b)
5
6     for i in range(n):
7         pivot = A[i][i]
8
9         # Normalize the pivot row
10        for j in range(n):
11            A[i][j] = A[i][j] / pivot
12            b[i] = b[i] / pivot
13
14        # Eliminate all other rows
15        for k in range(n):
16            if k != i:
17                factor = A[k][i]
18                for j in range(n):
19                    A[k][j] = A[k][j] - factor * A[i][j]
20                b[k] = b[k] - factor * b[i]
21
22    return b
23
24
25 A = [
26     [2.0, 3.0, 1.0],
27     [1.0, 2.0, 3.0],
28     [3.0, 1.0, 2.0]
29 ]
30 b = [9.0, 6.0, 8.0]
31
32 ans = gauss_jordan(A, b)
33 for i in range(len(ans)):
34     print(f"x{i+1} = {ans[i]:.4f}")

```

Program Execution Output & Convergence Table

```

x1 = 1.9444
x2 = 1.6111
x3 = 0.2778

```

2.3 Gauss-Jacobi Iterative Method

Category: Systems of Linear Algebraic Equations — **Code by:** Anushka — **Source:** 08_gauss_jacobi_detailed.

The Jacobi method is an iterative technique for solving $Ax = b$. In each step, each unknown x_i is updated solely using the values from the previous iteration step k . It converges for strictly diagonally dominant matrices.

Mathematical Theory & Formulation

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right) \quad (i = 1, 2, \dots, n)$$

$$\text{Strict Diagonal Dominance: } |a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 def gauss_jacobi(A, b, tol=1e-5, max_iter=50):
4     n = len(b)
5     x = []
6     for _ in range(n):
7         x.append(0.0)
8
9     # Print Table Header
10    header = f"{'Iter':<6}"
11    for i in range(n):
12        header += f"{'x' + str(i+1):<12}"
13    print(header)
14    print("-" * (6 + 12 * n))
15
16    for step in range(1, max_iter + 1):
17        x_new = []
18        for i in range(n):
19            sum_other = 0.0
20            for j in range(n):
21                if j != i:
22                    sum_other += A[i][j] * x[j]
23            xi_val = (b[i] - sum_other) / A[i][i]
24            x_new.append(xi_val)
25
26        # Print current iteration values
27        row_str = f"{'step':<6}"
28        for val in x_new:
29            row_str += f"{'val':<12.5f}"
30        print(row_str)
31
32        # Check convergence
33        max_diff = 0.0
34        for i in range(n):
35            diff = abs(x_new[i] - x[i])
36            if diff > max_diff:
37                max_diff = diff
38
39        if max_diff < tol:
40            return x_new
41
42        x = x_new
43
44    return x
45
46
47 A = [
48     [10.0, -1.0, 2.0],
49     [-1.0, 11.0, -1.0],
50     [2.0, -1.0, 10.0]
51 ]
52 b = [6.0, 25.0, -11.0]
53
54 sol = gauss_jacobi(A, b)
55 rounded_sol = []
56 for v in sol:
57     rounded_sol.append(round(v, 4))
58 print("\nSolution:", rounded_sol)

```

Program Execution Output & Convergence Table

Iter	x1	x2	x3
1	0.60000	2.27273	-1.10000
2	1.04727	2.22727	-0.99273
3	1.02127	2.27769	-1.08673
4	1.04511	2.26678	-1.07649
5	1.04197	2.26988	-1.08235
6	1.04346	2.26906	-1.08141
7	1.04319	2.26928	-1.08179
8	1.04328	2.26922	-1.08171
9	1.04326	2.26923	-1.08174
10	1.04327	2.26923	-1.08173

Solution: [1.0433, 2.2692, -1.0817]

2.4 Gauss-Seidel Iterative Method

Category: Systems of Linear Algebraic Equations — **Code by:** Anushka — **Source:** 09_gauss_seidel_detailed.

The Gauss-Seidel method accelerates Jacobi iteration by immediately utilizing the newly updated components $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$ as soon as they become available during the current iteration step.

Mathematical Theory & Formulation

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right)$$

Convergence Rate: Typically converges in roughly half the iterations of Jacobi.

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 def gauss_seidel(A, b, tol=1e-5, max_iter=50):
4     n = len(b)
5     x = []
6     for _ in range(n):
7         x.append(0.0)
8
9     header = f'{ "Iter":<6}'
10    for i in range(n):
11        header += f'{ "x' + str(i+1):<12}'
12    print(header)
13    print("-" * (6 + 12 * n))
14
15    for step in range(1, max_iter + 1):
16        max_diff = 0.0
17
18        for i in range(n):
19            sum_other = 0.0
20            for j in range(n):
21                if j != i:
22                    sum_other += A[i][j] * x[j]
23
24            x_new_val = (b[i] - sum_other) / A[i][i]
25            diff = abs(x_new_val - x[i])
26            if diff > max_diff:
27                max_diff = diff
28            x[i] = x_new_val
29
30        row_str = f'{step:<6}'
31        for val in x:
32            row_str += f'{val:<12.5f}'
33        print(row_str)
34
35        if max_diff < tol:
36            return x
37
38    return x
39
40
41 A = [
42     [10.0, -1.0, 2.0],
43     [-1.0, 11.0, -1.0],
44     [2.0, -1.0, 10.0]
45 ]
46 b = [6.0, 25.0, -11.0]
47
48 sol = gauss_seidel(A, b)
49 rounded_sol = []
50 for v in sol:
51     rounded_sol.append(round(v, 4))
52 print("\nSolution:", rounded_sol)

```

Program Execution Output & Convergence Table

Iter	x1	x2	x3
1	0.60000	2.32727	-0.98727
2	1.03018	2.27663	-1.07837
3	1.04334	2.26954	-1.08171
4	1.04330	2.26923	-1.08174
5	1.04327	2.26923	-1.08173
6	1.04327	2.26923	-1.08173

Solution: [1.0433, 2.2692, -1.0817]

Chapter 3

Interpolation & Polynomial Approximation

3.1 Lagrange Interpolation

Category: Interpolation & Polynomial Approximation — **Code by:** Anushka — **Source:** 10_lagrange_detailed.

Lagrange interpolation constructs the unique polynomial of degree at most n that passes through $n + 1$ given points (x_i, y_i) . The formulation does not require the data nodes to be evenly spaced.

Mathematical Theory & Formulation

$$P_n(x) = \sum_{i=0}^n y_i \ell_i(x), \quad \text{where } \ell_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

Python Implementation (Step-by-Step Loops & Pure Functions)

```
1 # Code by Anushka
2
3 def lagrange(x_vals, y_vals, target_x):
4     n = len(x_vals)
5     print(f'{i':<6}'Basis L_i(x)':<16}'y_i':<10}'Term':<12}")
6     print("-" * 44)
7
8     total = 0.0
9     for i in range(n):
10         # Calculate Lagrange basis polynomial L_i(x) using a loop
11         L_i = 1.0
12         for j in range(n):
13             if i != j:
14                 L_i = L_i * (target_x - x_vals[j]) / (x_vals[i] - x_vals[j])
15
16         term = y_vals[i] * L_i
17         total += term
18         print(f"{i:<6}{L_i:<16.5f}{y_vals[i]:<10.2f}{term:<12.5f}")
19
20     return total
21
22
23 x_pts = [5.0, 6.0, 9.0, 11.0]
24 y_pts = [12.0, 13.0, 14.0, 16.0]
25
26 ans = lagrange(x_pts, y_pts, 10.0)
27 print(f"\nInterpolated value: {ans:.5f}")
```

Program Execution Output & Convergence Table

i	Basis L_i(x)	y_i	Term
0	0.16667	12.00	2.00000
1	-0.33333	13.00	-4.33333
2	0.83333	14.00	11.66667
3	0.33333	16.00	5.33333

Interpolated value: 14.66667

3.2 Newton's Divided Difference Interpolation

Category: Interpolation & Polynomial Approximation — **Code by:** Anushka — **Source:** 11_newton_divided_diff

Newton's Divided Difference method constructs an interpolating polynomial using divided differences. It is versatile for unequally spaced data points and allows easy addition of new data points without recomputing the entire table.

Mathematical Theory & Formulation

$$f[x_i, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}$$

$$P_n(x) = f[x_0] + \sum_{k=1}^n f[x_0, \dots, x_k] \prod_{j=0}^{k-1} (x - x_j)$$

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 def newton_divided_diff(x, y, target_x):
4     n = len(x)
5
6     # Initialize 2D table with zeros using loops
7     table = []
8     for i in range(n):
9         row = []
10        for j in range(n):
11            row.append(0.0)
12        table.append(row)
13
14    # First column is y values
15    for i in range(n):
16        table[i][0] = y[i]
17
18    # Compute divided differences
19    for j in range(1, n):
20        for i in range(n - j):
21            table[i][j] = (table[i + 1][j - 1] - table[i][j - 1]) / (x[i + j] - x[i])
22
23    print("Divided Difference Table:")
24    for i in range(n):
25        row_str = f"{x[i]:<6.1f}"
26        for j in range(n - i):
27            row_str += f"{table[i][j]:<12.4f}"
28        print(row_str)
29
30    # Evaluate the polynomial at target_x
31    res = table[0][0]
32    prod_term = 1.0
33    for j in range(1, n):
34        prod_term = prod_term * (target_x - x[j - 1])
35        res += table[0][j] * prod_term
36
37    return res
38
39
40 x_pts = [4.0, 5.0, 7.0, 10.0, 11.0]
41 y_pts = [48.0, 100.0, 294.0, 900.0, 1210.0]
42
43 ans = newton_divided_diff(x_pts, y_pts, 6.0)
44 print(f"\nInterpolated Value: {ans:.5f}")

```

Program Execution Output & Convergence Table

```

Divided Difference Table:
4.0  48.0000  52.0000  15.0000  1.0000  0.0000
5.0  100.0000  97.0000  21.0000  1.0000
7.0  294.0000  202.0000  27.0000
10.0  900.0000  310.0000
11.0  1210.0000

Interpolated Value: 180.00000

```

3.3 Newton's Forward Difference Interpolation

Category: Interpolation & Polynomial Approximation — **Code by:** Anushka — **Source:** 12_newton_forward_det

Newton's Forward Difference formula is designed for tabulated data where nodes are equally spaced ($x_i = x_0 + ih$). It is ideal for interpolating values near the start of the table.

Mathematical Theory & Formulation

$$u = \frac{x - x_0}{h}, \quad \Delta y_i = y_{i+1} - y_i$$

$$P_n(x) = y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 + \cdots + \frac{u(u-1)\cdots(u-n+1)}{n!}\Delta^n y_0$$

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 import math
4
5 def newton_forward(x, y, target_x):
6     n = len(x)
7     h = x[1] - x[0]
8     u = (target_x - x[0]) / h
9
10    # Initialize forward difference table using loops
11    diff = []
12    for i in range(n):
13        row = []
14        for j in range(n):
15            row.append(0.0)
16        diff.append(row)
17
18    # Row 0 contains y values
19    for i in range(n):
20        diff[0][i] = y[i]
21
22    # Compute difference table
23    for order in range(1, n):
24        for i in range(n - order):
25            diff[order][i] = diff[order - 1][i + 1] - diff[order - 1][i]
26
27    print("Forward Difference Table:")
28    for i in range(n):
29        row_str = f"{x[i]:<6.1f}"
30        for j in range(n - i):
31            row_str += f"{diff[j][i]:<12.4f}"
32        print(row_str)
33
34    res = diff[0][0]
35    u_term = 1.0
36    for order in range(1, n):
37        u_term = u_term * (u - (order - 1))
38        res += (u_term * diff[order][0]) / math.factorial(order)
39
40    return res
41
42
43 x_pts = [10.0, 20.0, 30.0, 40.0, 50.0]
44 y_pts = [0.1736, 0.3420, 0.5000, 0.6428, 0.7660]
45
46 ans = newton_forward(x_pts, y_pts, 15.0)
47 print(f"\nInterpolated Value: {ans:.5f}")

```

Program Execution Output & Convergence Table

```

Forward Difference Table:
10.0 0.1736 0.1684 -0.0104 -0.0048 0.0004
20.0 0.3420 0.1580 -0.0152 -0.0044
30.0 0.5000 0.1428 -0.0196
40.0 0.6428 0.1232
50.0 0.7660

Interpolated Value: 0.25878

```

Chapter 4

Numerical Quadrature & Integration

4.1 Trapezoidal Rule of Numerical Integration

Category: Numerical Quadrature & Integration — **Code by:** Anushka — **Source:** 13_trapezoidal_detailed.py

The composite Trapezoidal rule evaluates $\int_a^b f(x) dx$ by approximating the integrand with linear segments across n uniform subintervals of width $h = \frac{b-a}{n}$.

Mathematical Theory & Formulation

$$I \approx \frac{h}{2} \left[f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right], \quad h = \frac{b-a}{n}$$

$$\text{Global Truncation Error: } E_T = -\frac{(b-a)h^2}{12} f''(\xi) \quad (\mathcal{O}(h^2) \text{ Accuracy})$$

Python Implementation (Step-by-Step Loops & Pure Functions)

```
1 # Code by Anushka
2
3 def trapezoidal(f, a, b, n=6):
4     h = (b - a) / float(n)
5
6     x_pts = []
7     for i in range(n + 1):
8         x_pts.append(a + i * h)
9
10    y_pts = []
11    for x in x_pts:
12        y_pts.append(f(x))
13
14    print(f"{'i':<6}{'x_i':<12}{'f(x_i)':<14}")
15    print("-" * 32)
16    for i in range(len(x_pts)):
17        print(f"{'i':<6}{'x_pts[i]':<12.4f}{'y_pts[i]':<14.5f}")
18
19    sum_middle = 0.0
20    for i in range(1, n):
21        sum_middle += y_pts[i]
22
23    integral = (h / 2.0) * (y_pts[0] + 2.0 * sum_middle + y_pts[n])
24    return integral
25
26
27 def f(x):
28     return 1.0 / (1.0 + x**2)
29
30
31 ans = trapezoidal(f, 0.0, 1.0, n=6)
32 print(f"\nIntegral: {ans:.5f}")
```

Program Execution Output & Convergence Table

i	x_i	f(x_i)
0	0.0000	1.00000
1	0.1667	0.97297
2	0.3333	0.90000
3	0.5000	0.80000
4	0.6667	0.69231
5	0.8333	0.59016
6	1.0000	0.50000

Integral: 0.78424

4.2 Simpson's 1/3 Rule of Numerical Integration

Category: Numerical Quadrature & Integration — **Code by:** Anushka — **Source:** 14_simpson_1_3_detailed.py

Simpson's 1/3 rule fits quadratic parabolas over adjacent pairs of subintervals. The total number of intervals n must be an even integer. It achieves fourth-order accuracy.

Mathematical Theory & Formulation

$$I \approx \frac{h}{3} \left[f(a) + f(b) + 4 \sum_{i \text{ odd}} f(x_i) + 2 \sum_{i \text{ even}} f(x_i) \right], \quad (n \text{ must be even})$$

$$\text{Global Truncation Error: } E_S = -\frac{(b-a)h^4}{180} f^{(4)}(\xi) \quad (\mathcal{O}(h^4) \text{ Accuracy})$$

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 def simpson_1_3(f, a, b, n=6):
4     if n % 2 != 0:
5         n += 1
6
7     h = (b - a) / float(n)
8
9     x_pts = []
10    for i in range(n + 1):
11        x_pts.append(a + i * h)
12
13    y_pts = []
14    for x in x_pts:
15        y_pts.append(f(x))
16
17    print(f"{'i':<6}{ 'x_i':<12}{ 'f(x_i)':<14}{ 'Mult':<6}")
18    print("-" * 38)
19    for i in range(len(x_pts)):
20        if i == 0 or i == n:
21            mult = 1
22        elif i % 2 != 0:
23            mult = 4
24        else:
25            mult = 2
26        print(f"{'i':<6}{x_pts[i]:<12.4f}{y_pts[i]:<14.5f}{mult:<6}")
27
28    sum_odd = 0.0
29    for i in range(1, n, 2):
30        sum_odd += y_pts[i]
31
32    sum_even = 0.0
33    for i in range(2, n, 2):
34        sum_even += y_pts[i]
35
36    integral = (h / 3.0) * (y_pts[0] + y_pts[n] + 4.0 * sum_odd + 2.0 * sum_even)
37    return integral
38
39
40 def f(x):
41     return 1.0 / (1.0 + x**2)
42
43
44 ans = simpson_1_3(f, 0.0, 1.0, n=6)
45 print(f"\nIntegral: {ans:.5f}")

```

Program Execution Output & Convergence Table

i	x_i	f(x_i)	Mult
0	0.0000	1.00000	1
1	0.1667	0.97297	4
2	0.3333	0.90000	2
3	0.5000	0.80000	4
4	0.6667	0.69231	2
5	0.8333	0.59016	4
6	1.0000	0.50000	1

Integral: 0.78540

4.3 Simpson's 3/8 Rule of Numerical Integration

Category: Numerical Quadrature & Integration — **Code by:** Anushka — **Source:** 15_simpson_3_8_detailed.py

Simpson's 3/8 rule fits cubic polynomials across groups of three consecutive subintervals. The total number of subintervals n must be a multiple of 3.

Mathematical Theory & Formulation

$$I \approx \frac{3h}{8} \left[f(a) + f(b) + 3 \sum_{i \nmid 3} f(x_i) + 2 \sum_{i|3} f(x_i) \right], \quad (n \in 3\mathbb{Z})$$

$$\text{Global Truncation Error: } E_{3/8} = -\frac{(b-a)h^4}{80} f^{(4)}(\xi) \quad (\mathcal{O}(h^4) \text{ Accuracy})$$

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 def simpson_3_8(f, a, b, n=6):
4     while n % 3 != 0:
5         n += 1
6
7     h = (b - a) / float(n)
8
9     x_pts = []
10    for i in range(n + 1):
11        x_pts.append(a + i * h)
12
13    y_pts = []
14    for x in x_pts:
15        y_pts.append(f(x))
16
17    print(f'{i':<6}{x_i':<12}{f(x_i)':<14}{Mult':<6}")
18    print("-" * 38)
19    for i in range(len(x_pts)):
20        if i == 0 or i == n:
21            mult = 1
22        elif i % 3 == 0:
23            mult = 2
24        else:
25            mult = 3
26        print(f'{i:<6}{x_pts[i]:<12.4f}{y_pts[i]:<14.5f}{mult:<6}")
27
28    sum_middle = 0.0
29    for i in range(1, n):
30        if i % 3 == 0:
31            sum_middle += 2.0 * y_pts[i]
32        else:
33            sum_middle += 3.0 * y_pts[i]
34
35    integral = (3.0 * h / 8.0) * (y_pts[0] + y_pts[n] + sum_middle)
36    return integral
37
38
39 def f(x):
40     return 1.0 / (1.0 + x**2)
41
42
43 ans = simpson_3_8(f, 0.0, 1.0, n=6)
44 print(f"\nIntegral: {ans:.5f}")

```

Program Execution Output & Convergence Table

i	x_i	f(x_i)	Mult
0	0.0000	1.00000	1
1	0.1667	0.97297	3
2	0.3333	0.90000	3
3	0.5000	0.80000	2
4	0.6667	0.69231	3
5	0.8333	0.59016	3
6	1.0000	0.50000	1

Integral: 0.78540

Chapter 5

Ordinary Differential Equations (ODEs)

5.1 Euler's Method for ODEs

Category: Ordinary Differential Equations (ODEs) — **Code by:** Anushka — **Source:** 16_euler_detailed.py

Euler's method is the classical first-order explicit single-step method for solving initial value problems $\frac{dy}{dx} = f(x, y)$ with $y(x_0) = y_0$.

Mathematical Theory & Formulation

$$y_{n+1} = y_n + h \cdot f(x_n, y_n)$$

Global Error Order: $\mathcal{O}(h)$ (First-Order Accurate)

Python Implementation (Step-by-Step Loops & Pure Functions)

```
1 # Code by Anushka
2
3 def euler(f, x0, y0, x_end, h=0.05):
4     x = x0
5     y = y0
6     step = 0
7
8     print(f'{Step':<6}{x':<10}{y':<12}{Slope':<12}")
9     print("-" * 40)
10
11     while x < x_end - 1e-9:
12         slope = f(x, y)
13         print(f'{step:<6}{x:<10.3f}{y:<12.5f}{slope:<12.5f}")
14         y = y + h * slope
15         x = x + h
16         step = step + 1
17
18     print(f'{step:<6}{x:<10.3f}{y:<12.5f}")
19     return y
20
21
22 def f(x, y):
23     return x + y
24
25
26 ans = euler(f, 0.0, 1.0, 0.2, h=0.05)
27 print(f"\ny(0.2): {ans:.5f}")
```

Program Execution Output & Convergence Table

Step	x	y	Slope
0	0.000	1.00000	1.00000
1	0.050	1.05000	1.10000
2	0.100	1.10500	1.20500
3	0.150	1.16525	1.31525
4	0.200	1.23101	

y(0.2): 1.23101

5.2 Modified Euler's (Heun's) Method

Category: Ordinary Differential Equations (ODEs) — **Code by:** Anushka — **Source:** 17_modified_euler_detail

Heun's (Modified Euler) method improves upon Euler's method by implementing a predictor-corrector scheme that averages slopes at both the start and predicted end of the step.

Mathematical Theory & Formulation

$$\text{Predictor: } \tilde{y}_{n+1} = y_n + h \cdot f(x_n, y_n)$$

$$\text{Corrector: } y_{n+1} = y_n + \frac{h}{2} [f(x_n, y_n) + f(x_{n+1}, \tilde{y}_{n+1})] \quad (\mathcal{O}(h^2) \text{ Accuracy})$$

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 def modified_euler(f, x0, y0, x_end, h=0.05):
4     x = x0
5     y = y0
6     step = 0
7
8     print(f'\'Step\':<6\{\'x\':<10\{\'y\':<12\{\'y_predict\':<14\}')
9     print("-" * 42)
10
11     while x < x_end - 1e-9:
12         k1 = f(x, y)
13         y_predict = y + h * k1
14         print(f"\'step:<6\{x:<10.3f\{y:<12.5f\{y_predict:<14.5f\}')
15         k2 = f(x + h, y_predict)
16         y = y + (h / 2.0) * (k1 + k2)
17         x = x + h
18         step = step + 1
19
20     print(f"\'step:<6\{x:<10.3f\{y:<12.5f\}')
21     return y
22
23
24 def f(x, y):
25     return x + y
26
27
28 ans = modified_euler(f, 0.0, 1.0, 0.2, h=0.05)
29 print(f"\ny(0.2): {ans:.5f}")

```

Program Execution Output & Convergence Table

Step	x	y	y_predict
0	0.000	1.00000	1.05000
1	0.050	1.05250	1.10763
2	0.100	1.11025	1.17077
3	0.150	1.17353	1.23971
4	0.200	1.24261	

y(0.2): 1.24261

5.3 Runge-Kutta 4th Order Method (RK4)

Category: Ordinary Differential Equations (ODEs) — **Code by:** Anushka — **Source:** 18_rk4_detailed.py

The classical Runge-Kutta 4th Order (RK4) method evaluates four intermediate slopes per step and combines them with Simpson-type weighting to achieve fourth-order accuracy without evaluating higher derivatives.

Mathematical Theory & Formulation

$$\begin{aligned}
 k_1 &= hf(x_n, y_n), \quad k_2 = hf\left(x_n + \frac{h}{2}, y_n + \frac{k_1}{2}\right) \\
 k_3 &= hf\left(x_n + \frac{h}{2}, y_n + \frac{k_2}{2}\right), \quad k_4 = hf(x_n + h, y_n + k_3) \\
 y_{n+1} &= y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (\mathcal{O}(h^4) \text{ Accuracy})
 \end{aligned}$$

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 def rk4(f, x0, y0, x_end, h=0.05):
4     x = x0
5     y = y0
6     step = 0
7
8     print(f"{'Step':<6}{'x':<8}{'y':<10}{'k1':<9}{'k2':<9}{'k3':<9}{'k4':<9}")
9     print("-" * 60)
10
11     while x < x_end - 1e-9:
12         k1 = h * f(x, y)
13         k2 = h * f(x + h / 2.0, y + k1 / 2.0)
14         k3 = h * f(x + h / 2.0, y + k2 / 2.0)
15         k4 = h * f(x + h, y + k3)
16         print(f"{step:<6}{x:<8.2f}{y:<10.4f}{k1:<9.4f}{k2:<9.4f}{k3:<9.4f}{k4:<9.4f}")
17         y = y + (k1 + 2.0 * k2 + 2.0 * k3 + k4) / 6.0
18         x = x + h
19         step = step + 1
20
21     print(f"{step:<6}{x:<8.2f}{y:<10.4f}")
22     return y
23
24
25 def f(x, y):
26     return x + y
27
28
29 ans = rk4(f, 0.0, 1.0, 0.2, h=0.05)
30 print(f"\ny(0.2): {ans:.5f}")

```

Program Execution Output & Convergence Table

Step	x	y	k1	k2	k3	k4
0	0.00	1.0000	0.0500	0.0525	0.0526	0.0551
1	0.05	1.0525	0.0551	0.0578	0.0578	0.0605
2	0.10	1.1103	0.0605	0.0633	0.0633	0.0662
3	0.15	1.1737	0.0662	0.0691	0.0692	0.0721
4	0.20	1.2428				

y(0.2): 1.24281

Chapter 6

Eigenvalue Problems & Curve Fitting

6.1 Power Method for Dominant Eigenvalue

Category: [Eigenvalues & Curve Fitting](#) — **Code by:** [Anushka](#) — **Source:** `19_power_method_detailed.py`

The Power Method is an iterative technique that computes the dominant eigenvalue (largest absolute magnitude) and its corresponding eigenvector for a square matrix A .

Mathematical Theory & Formulation

$$y_{k+1} = Ax_k, \quad \lambda_{k+1} = \max(|y_{k+1}|), \quad x_{k+1} = \frac{y_{k+1}}{\lambda_{k+1}}$$

$$\text{Convergence Rate: } \left| \frac{\lambda_2}{\lambda_1} \right|^k \quad (\text{Requires } |\lambda_1| > |\lambda_2|)$$

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 def power_method(A, tol=1e-5, max_iter=50):
4     n = len(A)
5     x = []
6     for _ in range(n):
7         x.append(1.0)
8     lambda_old = 0.0
9
10    print(f"{'Iter':<6}{'Eigenvalue':<14}{'Eigenvector':<20}")
11    print("-" * 40)
12
13    for step in range(1, max_iter + 1):
14        # Matrix-vector multiplication y = A * x
15        y = []
16        for i in range(n):
17            row_sum = 0.0
18            for j in range(n):
19                row_sum += A[i][j] * x[j]
20            y.append(row_sum)
21
22        # Find maximum absolute value in y
23        lambda_new = y[0]
24        for val in y:
25            if abs(val) > abs(lambda_new):
26                lambda_new = val
27
28        # Normalize eigenvector x = y / lambda_new
29        for i in range(n):
30            x[i] = y[i] / lambda_new
31
32        vec_str = "["
33        for i in range(n):
34            vec_str += f"{x[i]:.3f}"
35            if i < n - 1:
36                vec_str += ", "
37        vec_str += "]"
38
39        print(f"{step:<6}{lambda_new:<14.4f}{vec_str:<20}")
40
41        if abs(lambda_new - lambda_old) < tol:
42            return lambda_new, x
43
44        lambda_old = lambda_new
45
46    return lambda_new, x
47
48 matrix = [
49     [4.0, 1.0],
50     [2.0, 3.0]
51 ]
52
53
54 val, vec = power_method(matrix)
55 rounded_vec = []
56 for v in vec:
57     rounded_vec.append(round(v, 4))
58 print(f"\nEigenvalue: {val:.4f}, Eigenvector: {rounded_vec}")

```

Program Execution Output & Convergence Table

Iter	Eigenvalue	Eigenvector
1	5.0000	[1.000, 1.000]
2	5.0000	[1.000, 1.000]

Eigenvalue: 5.0000, Eigenvector: [1.0, 1.0]

6.2 Linear Curve Fitting (Least Squares)

Category: Eigenvalues & Curve Fitting — **Code by:** Anushka — **Source:** 20_linear_fit_detailed.py

Linear curve fitting finds the best-fit line $y = mx + c$ for a dataset of n points by minimizing the sum of squared vertical residuals $\sum (y_i - (mx_i + c))^2$.

Mathematical Theory & Formulation

$$\text{Normal Equations: } nc + m \sum x = \sum y, \quad c \sum x + m \sum x^2 = \sum xy$$

$$m = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2}, \quad c = \frac{\sum y - m \sum x}{n}$$

Python Implementation (Step-by-Step Loops & Pure Functions)

```

1 # Code by Anushka
2
3 def linear_fit(x, y):
4     n = len(x)
5
6     # Compute summations using loops
7     sx = 0.0
8     sy = 0.0
9     sxy = 0.0
10    sx2 = 0.0
11
12    for i in range(n):
13        sx += x[i]
14        sy += y[i]
15        sxy += x[i] * y[i]
16        sx2 += x[i] ** 2
17
18    print(f"n = {n}, sum_x = {sx:.2f}, sum_y = {sy:.2f}, sum_xy = {sxy:.2f}, sum_x2 = {sx2:.2f}")
19
20    m = (n * sxy - sx * sy) / (n * sx2 - sx**2)
21    c = (sy - m * sx) / float(n)
22    return m, c
23
24
25 x_data = [1.0, 2.0, 3.0, 4.0, 5.0]
26 y_data = [2.2, 2.8, 3.6, 4.5, 5.1]
27
28 m, c = linear_fit(x_data, y_data)
29 print(f"Fitted Line: y = {m:.4f}x + {c:.4f}")

```

Program Execution Output & Convergence Table

```

n = 5, sum_x = 15.00, sum_y = 18.20, sum_xy = 62.10, sum_x2 = 55.00
Fitted Line: y = 0.7500x + 1.3900

```

Chapter 7

Master Summary & Quick-Reference Guide

Unit 1: Roots of Non-Linear Equations

- **1. Bisection Method:** $c = (a + b)/2.0$. Halves bracket interval. Order: Linear ($\mathcal{O}(2^{-n})$). Always converges if $f(a)f(b) < 0$.
- **2. Regula-Falsi (False Position):** $c = \frac{af(b)-bf(a)}{f(b)-f(a)}$. Secant line x -intercept with guaranteed bracketing. Order: Superlinear to Linear.
- **3. Newton-Raphson Method:** $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$. Tangent line root approximation. Order: Quadratic ($p = 2$).
- **4. Secant Method:** $x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}$. Finite-difference slope approximation. Order: $p \approx 1.618$.
- **5. Fixed-Point Iteration:** $x_{k+1} = g(x_k)$. Converges linearly if $|g'(x)| < 1$ in neighborhood of the fixed point.

Unit 2: Systems of Linear Algebraic Equations

- **6. Gauss Elimination:** Converts $[A \mid b]$ to upper-triangular matrix, followed by back substitution. Operations: $\mathcal{O}(\frac{2}{3}n^3)$.
- **7. Gauss-Jordan Elimination:** Direct reduction to reduced row echelon form $[I \mid x]$ eliminating back substitution. Operations: $\mathcal{O}(n^3)$.
- **8. Gauss-Jacobi Iteration:** $x_i^{(k+1)} = \frac{1}{a_{ii}}(b_i - \sum_{j \neq i} a_{ij}x_j^{(k)})$. Requires strict diagonal dominance.
- **9. Gauss-Seidel Iteration:** $x_i^{(k+1)} = \frac{1}{a_{ii}}(b_i - \sum_{j < i} a_{ij}x_j^{(k+1)} - \sum_{j > i} a_{ij}x_j^{(k)})$. Converges $\approx 2\times$ faster than Jacobi.

Unit 3: Interpolation & Polynomial Approximation

- **10. Lagrange Interpolation:** $P(x) = \sum y_i \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$. Flexible formulation for arbitrary, unequally spaced nodes.
- **11. Newton's Divided Difference:** $P(x) = f[x_0] + \sum_{k=1}^n f[x_0, \dots, x_k] \prod_{j=0}^{k-1} (x - x_j)$. Ideal for incrementally adding data points.
- **12. Newton's Forward Difference:** $P(x) = y_0 + u\Delta y_0 + \frac{u(u-1)}{2!}\Delta^2 y_0 + \dots$. Specifically for equispaced points ($u = (x - x_0)/h$).

Unit 4: Numerical Quadrature & Integration

- **13. Trapezoidal Rule:** $I \approx \frac{h}{2}[f(a) + 2\sum_{i=1}^{n-1} f(x_i) + f(b)]$. Linear piecewise approximation. Error: $\mathcal{O}(h^2)$.
- **14. Simpson's 1/3 Rule:** $I \approx \frac{h}{3}[f(a) + f(b) + 4\sum_{\text{odd}} + 2\sum_{\text{even}}]$. Parabolic piecewise fits. Requires even n . Error: $\mathcal{O}(h^4)$.
- **15. Simpson's 3/8 Rule:** $I \approx \frac{3h}{8}[f(a) + f(b) + 3\sum_{i \nmid 3} + 2\sum_{i|3}]$. Cubic piecewise fits. Requires n multiple of 3. Error: $\mathcal{O}(h^4)$.

Unit 5 & 6: Differential Equations, Eigenvalues & Curve Fitting

- **16. Euler's Method:** $y_{n+1} = y_n + hf(x_n, y_n)$. Single-step explicit tangent method. Order: First-order $\mathcal{O}(h)$.
- **17. Modified Euler (Heun's):** Predictor-corrector scheme. Predicts $\tilde{y}_{n+1}$, corrects using average slope. Order: Second-order $\mathcal{O}(h^2)$.
- **18. Runge-Kutta 4 (RK4):** Four weighted intermediate slopes per step. Gold standard for initial value ODEs. Order: Fourth-order $\mathcal{O}(h^4)$.
- **19. Power Method:** $y_{k+1} = Ax_k$, $\lambda_{k+1} = \max(|y_{k+1}|)$, $x_{k+1} = y_{k+1}/\lambda_{k+1}$. Iteratively converges to dominant eigenvalue.
- **20. Linear Curve Fitting:** Minimizes $\sum (y_i - (mx_i + c))^2$ to determine optimal slope m and intercept c for best-fit straight line.